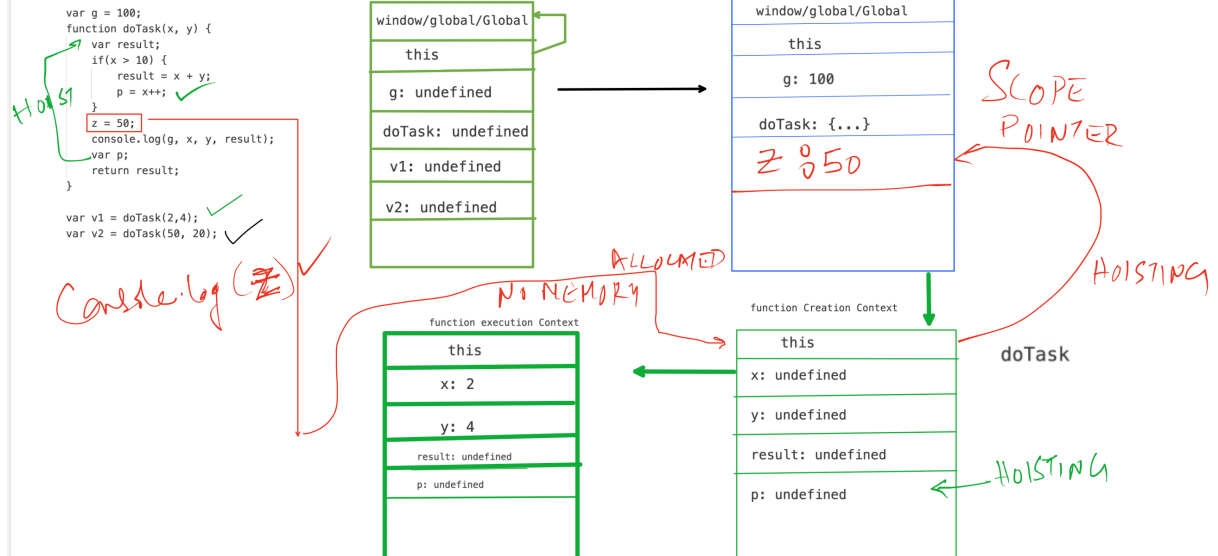


MEMORY ALLOCATED



```

console.log("Hello");
setTimeout(function timer1() {
  console.log("T1");
}, 0);

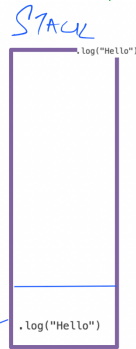
setInterval(function timer2() {
  console.log("T2");
}, 10);

Promise.resolve("Task Complete").then(function handler(data) {
  console.log(data);
});

Promise.resolve("Second Task").then(function handler2(data) {
  console.log(data);
});

console.log("Bye");

```



Hello

```

console.log("Hello");
setTimeout(function timer1() {
  console.log("T1");
}, 0);

setInterval(function timer2() {
  console.log("T2");
}, 10);

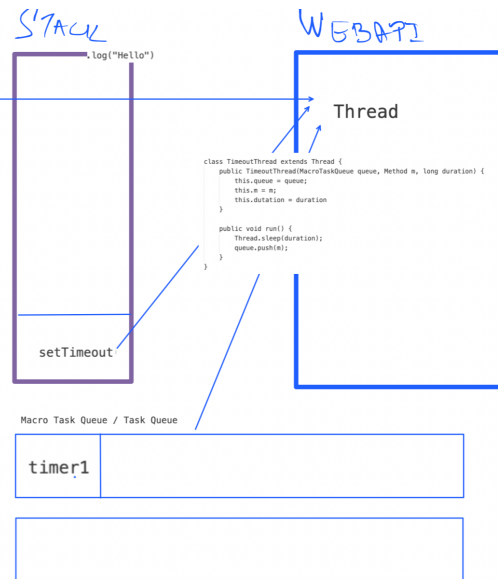
Promise.resolve("Task Complete").then(function handler(data) {
  console.log(data);
})

Promise.resolve("Second Task").then(function handler2(data) {
  console.log(data);
});

console.log("Bye");

```

Hello



```

console.log("Hello");
setTimeout(function timer1() {
  console.log("T1");
}, 0);

setInterval(function timer2() {
  console.log("T2");
}, 10);

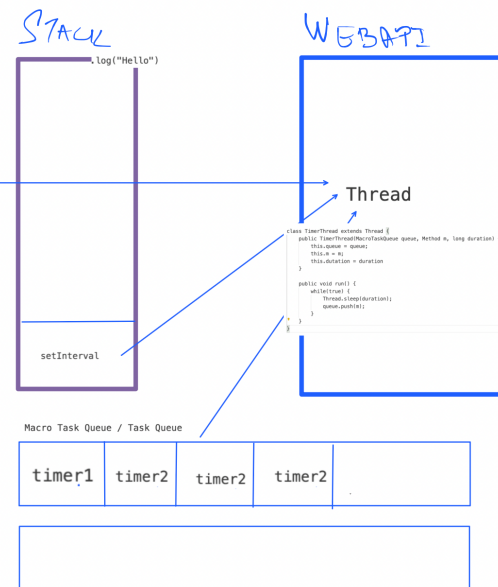
Promise.resolve("Task Complete").then(function handler(data) {
  console.log(data);
})

Promise.resolve("Second Task").then(function handler2(data) {
  console.log(data);
});

console.log("Bye");

```

Hello



```

console.log("Hello");
setTimeout(function timer1() {
  console.log("T1");
}, 0);

setInterval(function timer2() {
  console.log("T2");
}, 10);

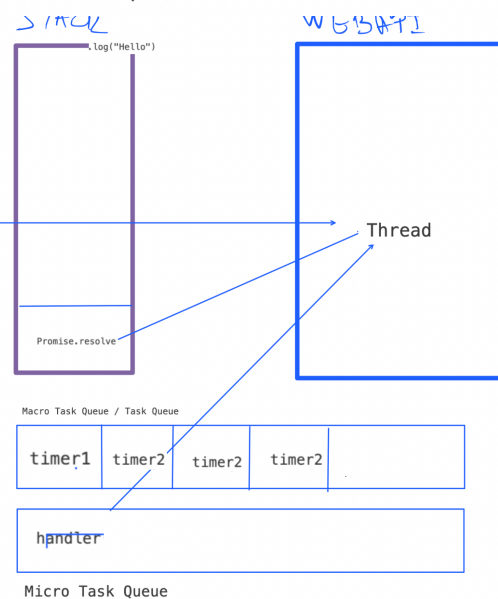
Promise.resolve("Task Complete").then(function handler(data) {
  console.log(data);
})

Promise.resolve("Second Task").then(function handler2(data) {
  console.log(data);
});

console.log("Bye");

```

Hello



```

console.log("Hello");
setTimeout(function timer1() {
  console.log("T1");
}, 0);

setInterval(function timer2() {
  console.log("T2");
}, 10);

Promise.resolve("Task Complete").then(function h_log("Bye")data) {
  console.log(data);
})

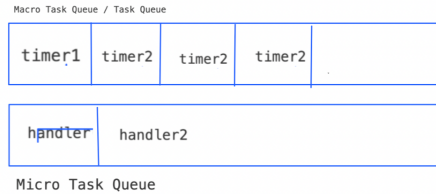
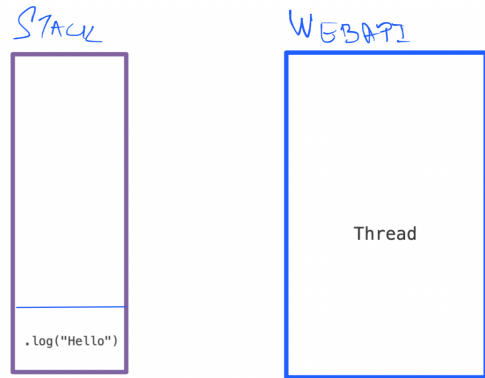
Promise.resolve("Second Task").then(function handler2(data) {
  console.log(data);
});

console.log("Bye");

```

Hello
 Bye

↓ 1st Tick



```

class EventLoop extends Thread {
  public EventLoop(Stack stack, MacroTaskQueue q1, MicroTaskQueue q2) {
  }

  public void run() {
    while(true && stack.isEmpty()) {
      for(Method m : q2.methods()) {
        Method m = q2.pop();
        if(stack.isEmpty()) {
          stack.push(m);
        }
      }
      for(Method m : q1.methods()) {
        Method m = q1.pop();
        if(stack.isEmpty()) {
          stack.push(m);
        }
      }
    }
  }
}

```

```

console.log("Hello");
setTimeout(function timer1() {
  console.log("T1");
}, 0);

setInterval(function timer2() {
  console.log("T2");
}, 10);

Promise.resolve("Task Complete").then(function handler(data) {
  console.log(data);
})

Promise.resolve("Second Task").then(function handler2(data) {
  console.log(data);
});

console.log("Bye");

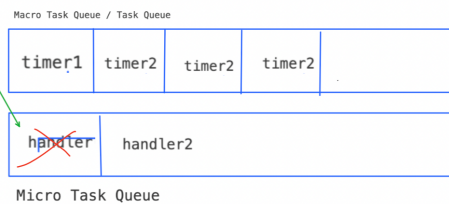
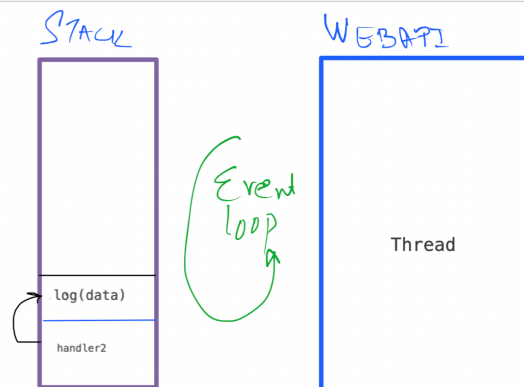
```

```

class EventLoop extends Thread {
  public EventLoop(Stack stack, MacroTaskQueue q1, MicroTaskQueue q2) {
  }

  public void run() {
    while(true && stack.isEmpty()) {
      for(Method m : q2.methods()) {
        Method m = q2.pop();
        if(stack.isEmpty()) {
          stack.push(m);
        }
      }
      for(Method m : q1.methods()) {
        Method m = q1.pop();
        if(stack.isEmpty()) {
          stack.push(m);
        }
      }
    }
  }
}

```



Hello
 Bye

Task Complete
 Second Task

```

console.log("Hello");
setTimeout(function timer1() {
  console.log("T1");
}, 0);

setInterval(function timer2() {
  console.log("T2");
}, 10);

Promise.resolve("Task Complete").then(function handler(data) {
  console.log(data);
})

Promise.resolve("Second Task").then(function handler2(data) {
  console.log(data);
});

console.log("Bye");

```

```

class EventLoop extends Thread {
  public EventLoop(Stack stack, MacroTaskQueue q1, MicroTaskQueue q2) {
  }

  public void run() {
    while(true && stack.isEmpty()) {
      for(Method m : q2.methods()) {
        Method m = q2.pop();
        if(stack.isEmpty()) {
          stack.push(m);
        }
      }
      for(Method m : q1.methods()) {
        Method m = q1.pop();
        if(stack.isEmpty()) {
          stack.push(m);
        }
      }
    }
  }
}

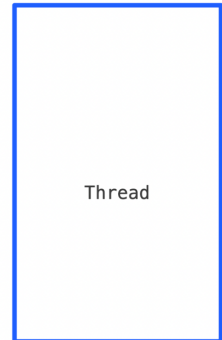
```

Hello
 Bye
 Task Complete
 Second Task
 'T1'

STACK

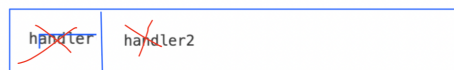


WEBAPI



Event loop

Macro Task Queue / Task Queue



Micro Task Queue